

# MySQL DELETE utasítás

## Mi az DELETE utasítás?

A DELETE utasítással rekordokat (sorokat) törölhetünk egy táblából. Ez az egyik legfontosabb és egyben legveszélyesebb DML (Data Manipulation Language) utasítás, mivel a törlés általában **visszavonhatatlan**.

## Alapszintaxis

```
DELETE FROM tabla_nev  
WHERE feltetel;
```

Szintaxis elemei:

- **DELETE FROM tabla\_nev** - megadjuk, melyik táblából szeretnénk törölni
- **WHERE** - feltétel, amely meghatározza, mely sorok legyenek törölve
- ⚠ **KRITIKUS:** Ha elhagyjuk a WHERE feltételt, MINDEN sor törlődik a táblából!

## Példák egyszerű DELETE utasításokra

### 1. Egy konkrét rekord törlése

```
-- Egy adott ID-jú film törlése  
DELETE FROM filmek  
WHERE id = 42;
```

### 2. Több rekord törlése egyszerű feltétel alapján

```
-- Az összes 1950 előtti film törlése  
DELETE FROM filmek  
WHERE ev < 1950;
```

### 3. Több feltétel alapján történő törlés

```
-- Azok a könyvek törlése, amelyek olcsóbbak 1000 Ft-nál ÉS nincs belőlük  
készleten  
DELETE FROM konyvek  
WHERE ar < 1000 AND keszlet = 0;
```

### 4. NULL értékek alapján történő törlés

```
-- Ügyfelek törlése, akiknek nincs megadva email címük
DELETE FROM ugyfelek
WHERE email IS NULL;

-- Üres string NEM ugyanaz, mint a NULL!
DELETE FROM ugyfelek
WHERE email = '' OR email IS NULL;
```

## 5. Szöveges érték alapján történő törlés

```
-- Egy adott nevű felhasználó törlése
DELETE FROM felhasznalok
WHERE felhasznalo_nev = 'teszt_user';

-- Részleges egyezés (LIKE operátor)
DELETE FROM termek
WHERE nev LIKE '%teszt%';
```

## 6. Dátum alapján történő törlés

```
-- 2020 előtti rendelések törlése
DELETE FROM rendelések
WHERE rendeles_datum < '2020-01-01';

-- Egy évnél régebbi naplóbejegyzések törlése
DELETE FROM log_tabla
WHERE datum < DATE_SUB(CURDATE(), INTERVAL 1 YEAR);
```

## Speciális DELETE esetek

### 1. IN operátor használata

```
-- Több konkrét érték alapján törlés
DELETE FROM tanulok
WHERE osztaly IN ('9.A', '9.B', '9.C');
```

### 2. NOT IN használata

```
-- Azok törlése, amik NINCSENEK egy listában
DELETE FROM termek
WHERE kategoria_id NOT IN (1, 5, 10, 15);
```

### 3. JOIN használata DELETE-ben (MySQL specifikus szintaxis)

```
-- Kapcsolódó tábla alapján törlés
DELETE k
FROM konyvek k
INNER JOIN szerzok sz ON k.szerzo_id = sz.id
WHERE sz.orszag = 'Magyarország' AND k.ev < 2000;

-- Több táblából egyidejű törlés
DELETE k, r
FROM kosar_tetelek k
INNER JOIN rendelések r ON k.rendeles_id = r.id
WHERE r.statusz = 'torolve';
```

### 4. LIMIT használata

```
-- Csak az első 10 rekord törlése
DELETE FROM log_tabla
WHERE datum < '2023-01-01'
LIMIT 10;

-- Hasznos nagy mennyiségű adat törlésénél (részletekben)
DELETE FROM nagy_tabla
WHERE feltétel
LIMIT 1000;
```

## TRUNCATE vs DELETE

### DELETE

```
-- Feltétellel törli a sorokat
DELETE FROM tabla_nev
WHERE feltétel;
```

#### Jellemzők:

- Törölhető részben (WHERE feltétellel)
- Lassabb (soronként dolgozza fel)
- Naplózza a műveleteket (visszavonható ROLLBACK-kel)
- Triggereket aktiválja
- AUTO\_INCREMENT értéket nem állítja vissza

### TRUNCATE

```
-- Az ÖSSZES sort törli (nincs WHERE)
TRUNCATE TABLE tabla_nev;
```

### Jellemzők:

- Mindig az ÖSSZES sort törli
- Gyorsabb (a teljes táblát törli és újraépíti)
- Nem naplózza a sorokat egyenként
- Triggereket NEM aktiválja
- AUTO\_INCREMENT értéket 1-re állítja vissza
- Nem lehet visszavonni (ROLLBACK)

### Mikor melyiket használd?

```
-- Ha feltételesen törölsz: DELETE
DELETE FROM log_tabla
WHERE datum < '2023-01-01';

-- Ha az EGÉSZ táblát üríteni akarod: TRUNCATE
TRUNCATE TABLE temp_adatok;
-- Ez gyorsabb és tisztább, mint:
DELETE FROM temp_adatok; -- Ne használd tábla ürítésre!
```

## Biztonságos DELETE gyakorlat

### 1. Mindig először ellenőrizd SELECT-tel!

```
-- ELŐBB futtasd le SELECT-tel:
SELECT * FROM filmek WHERE ev < 1950;
-- Számold meg, hány sor érintett:
SELECT COUNT(*) FROM filmek WHERE ev < 1950;

-- HA rendben van, CSAK AKKOR törölj:
DELETE FROM filmek
WHERE ev < 1950;
```

### 2. Használj tranzakciókat!

```
START TRANSACTION;

DELETE FROM rendelések
WHERE statusz = 'törölve' AND datum < '2020-01-01';

-- Ellenőrizd, hány sort töröltél
SELECT ROW_COUNT(); -- Visszaadja a törölt sorok számát
```

```
-- Ha rendben van:  
COMMIT;  
-- Ha mégsem jó:  
ROLLBACK;
```

### 3. Készíts biztonsági mentést!

```
-- Biztonsági mentés készítése törlés előtt  
CREATE TABLE filmek_backup AS  
SELECT * FROM filmek  
WHERE ev < 1950;  
  
-- Most biztonságosan törölhetsz  
DELETE FROM filmek  
WHERE ev < 1950;  
  
-- Ha probléma van, visszaállíthatod:  
-- INSERT INTO filmek SELECT * FROM filmek_backup;
```

### 4. LIMIT használata teszteléskor

```
-- Először csak 1 rekordot törölj tesztként  
DELETE FROM termek  
WHERE kategoria = 'teszt'  
LIMIT 1;  
  
-- Ha rendben van, töröld a többi is  
DELETE FROM termek  
WHERE kategoria = 'teszt';
```

## DELETE és külső kulcsok (Foreign Keys)

### ON DELETE CASCADE

Ha a táblában CASCADE van beállítva, a kapcsolódó rekordok is törlődnek:

```
-- Tábla definíció:  
CREATE TABLE rendelések (  
    id INT PRIMARY KEY,  
    ugyfel_id INT,  
    FOREIGN KEY (ugyfel_id) REFERENCES ugyfelek(id)  
    ON DELETE CASCADE  
);  
  
-- Ha törlöd az ügyfelet, a rendelései is törlődnek:
```

```
DELETE FROM ugyfelek WHERE id = 5;  
-- Ez automatikusan törli az összes rendelést is, ahol ugyfel_id = 5
```

## ON DELETE SET NULL

```
CREATE TABLE rendelesek (  
    id INT PRIMARY KEY,  
    ugyfel_id INT,  
    FOREIGN KEY (ugyfel_id) REFERENCES ugyfelek(id)  
        ON DELETE SET NULL  
);  
  
-- Ha törölöd az ügyfelet, a rendelések ugyfel_id mezője NULL lesz:  
DELETE FROM ugyfelek WHERE id = 5;  
-- A rendelések megmaradnak, de ugyfel_id = NULL lesz bennük
```

## ON DELETE RESTRICT (alapértelmezett)

```
-- Ha RESTRICT van beállítva (vagy nincs megadva ON DELETE):  
DELETE FROM ugyfelek WHERE id = 5;  
-- Error: Cannot delete or update a parent row: a foreign key constraint fails  
  
-- Megoldás: Először a kapcsolódó rekordokat kell törölni  
DELETE FROM rendelesek WHERE ugyfel_id = 5;  
DELETE FROM ugyfelek WHERE id = 5;
```

## Gyakori hibák és megoldások

### ✗ Hiba: WHERE feltétel nélküli DELETE

```
-- VESZÉLYES! Az ÖSSZES sor törlődik!  
DELETE FROM tanulok;
```

#### Megoldás:

- Mindig használj WHERE feltételt!
- Ha az egész táblát akarod üríteni, inkább használj TRUNCATE-et!

### ✗ Hiba: Safe Update Mode

MySQL Workbench-ben alapértelmezetten be van kapcsolva a "Safe Update" mód.

#### Hibaüzenet:

```
Error Code: 1175. You are using safe update mode and you tried to delete
from a table without a WHERE that uses a KEY column.
```

**Megoldás 1:** Kapcsold ki ideiglenesen:

```
SET SQL_SAFE_UPDATES = 0;
-- DELETE utasítás
SET SQL_SAFE_UPDATES = 1;
```

**Megoldás 2:** Használj kulcs oszlopot a WHERE feltételben:

```
DELETE FROM tanulok
WHERE id IN (1, 2, 3); -- id az elsődleges kulcs
```

✗ Hiba: Foreign Key megszorítás

```
DELETE FROM kategoriak WHERE id = 5;
-- Error: Cannot delete or update a parent row: a foreign key constraint fails
```

**Megoldás:** Először a hivatkozó rekordokat kell törölni:

```
-- 1. Töröld a kapcsolódó termékeket
DELETE FROM termek WHERE categoria_id = 5;
-- 2. Most már törölheted a kategóriát
DELETE FROM kategoriak WHERE id = 5;
```

✗ Hiba: Rossz feltétel (OR vs AND)

```
-- Ez NEM azt törli, amit szeretnél!
DELETE FROM termek
WHERE nev = 'Laptop' OR ar > 100000;
-- Ez minden laptopot ÉS minden 100000 Ft feletti terméket töröl!

-- Valószínűleg ezt szeretted volna:
DELETE FROM termek
WHERE nev = 'Laptop' AND ar > 100000;
```

## Teljesítmény és optimalizálás

### Indexek használata

A DELETE gyorsabb, ha indexelt oszlopon van a WHERE feltétel:

```
-- Gyors, mert 'id' indexelt (PRIMARY KEY)
DELETE FROM rendelesek
WHERE id = 1000;

-- Lassabb lehet, ha 'datum' nincs indexelve
DELETE FROM rendelesek
WHERE datum < '2020-01-01';
```

Nagy mennyiségű adat törlése

```
-- Rossz megközelítés (egy lépésben):
DELETE FROM log_tabla
WHERE datum < '2020-01-01';
-- Ez lefagyaszthatja az adatbázist!

-- Jó megközelítés (részletekben):
DELETE FROM log_tabla
WHERE datum < '2020-01-01'
LIMIT 1000;
-- Ismételd meg, amíg 0 sor nem törlődik
```

## Gyakorlati feladatok

### 1. feladat

Töröld az összes olyan könyvet, amelynek az ára 0 vagy NULL!

► Megoldás

```
DELETE FROM konyvek
WHERE ar = 0 OR ar IS NULL;
```

### 2. feladat

Töröld azokat a rendeléseket, amelyek 2022-ben érkeztek és a státuszuk "törölve"!

► Megoldás

```
DELETE FROM rendelesek
WHERE YEAR(rendeles_datum) = 2022
AND statusz = 'törölve';
```



# DELETE vs DROP vs TRUNCATE összehasonlítás

| Jellemző           | DELETE                                              | TRUNCATE                                          | DROP                                              |
|--------------------|-----------------------------------------------------|---------------------------------------------------|---------------------------------------------------|
| Mit töröl?         | Sorokat (rekordokat)                                | Összes sort                                       | Teljes táblát                                     |
| WHERE használható? | <input checked="" type="checkbox"/> Igen            | <input checked="" type="checkbox"/> Nem           | <input checked="" type="checkbox"/> Nem           |
| Visszavonható?     | <input checked="" type="checkbox"/> Igen (ROLLBACK) | <input checked="" type="checkbox"/> Nem           | <input checked="" type="checkbox"/> Nem           |
| Sebesség           | Lassú                                               | Gyors                                             | Gyors                                             |
| AUTO_INCREMENT     | Nem állítja vissza                                  | Visszaállítja                                     | N/A                                               |
| Triggerek          | <input checked="" type="checkbox"/> Aktiválja       | <input checked="" type="checkbox"/> Nem aktiválja | <input checked="" type="checkbox"/> Nem aktiválja |
| Táblastruktúra     | Megmarad                                            | Megmarad                                          | Törlődik                                          |

```
-- DELETE - Sorokat töröl feltétellel
DELETE FROM tabla WHERE feltetel;

-- TRUNCATE - Minden sort töröl (gyorsabb)
TRUNCATE TABLE tabla;

-- DROP - Teljes táblát töröl (struktúrával együtt!)
DROP TABLE tabla;
```

## Összefoglalás

A DELETE utasítás használatakor alapelvek:

- ☒ **Mindig használj WHERE feltételt** (kivéve tudatos teljes törlés)
- ☒ **Először SELECT-tel ellenőrizz**, utána törölj
- ☒ **Készíts biztonsági mentést** kritikus adatok törlésekor
- ☒ **Figyelj a külső kulcsokra** és ON DELETE beállításokra
- ☒ **Használj LIMIT-et** teszteléskor és nagy mennyiségű adat törlésekor
- ☒ **TRUNCATE a gyorsabb** ha az egész táblát üríteni akarod

### ⚠ VESZÉLY ZÓNÁK:

- ☒ **DELETE FROM tabla;** - Minden törlődik!
- ☒ **DELETE FROM tabla WHERE feltetel;** - Ellenőrizd kétszer is a feltételt!
- ☒ **CASCADE** beállításoknál - Tudatában kell lenni, mi minden törlődik!

**Arany szabály:** A DELETE művelet általában VISSZAVONHATATLAN! Gondold át kétszer, mielőtt végrehajtod!

**Biztonsági tipp:** Kritikus adatok esetén használj "soft delete" megoldást (törlés helyett egy **torolve** flag beállítása):

```
-- Fizikai törlés helyett (NEM javasolt kritikus adatnál):  
-- DELETE FROM ugyfelek WHERE id = 5;  
  
-- Logikai törlés (soft delete - JAVASOLT):  
UPDATE ugyfelek  
SET aktiv = 0, torles_datum = CURDATE()  
WHERE id = 5;
```